

DESIGN AND ANALYSIS ALGORITHMS

LECTURE 6

Greedy Methods (Phương pháp tham lam)

Review the Previous Lesson

- Trial and error method
 - Brute force / Exhaustive searching
 - Backtracking
 - Branch and bound

Lecture outline

- ❑ Introduction
- ❑ Greedy strategy
- ❑ Method template and diagram
- ❑ Examples
 - Change Coins
 - Activity Selection Problem
 - Minimal Spanning Tree

Reference link and book chapter

Prof. Xukai Zou, Computer Science Dept.,
Indiana University Purdue University Indianapolis

<http://cs.iupui.edu/~xzou/>

Prof. Pasi Fränti, School of Computing,
University of Eastern Finland

<http://cs.uef.fi/pages/franti/asa/>

Anany's book, chapter 9

Pre-words

- “ ‘Greed,’ for lack of a better word, is good!”
Michael Douglas, actor in the role Gordon Gecko,
Wall Street film, 1987.



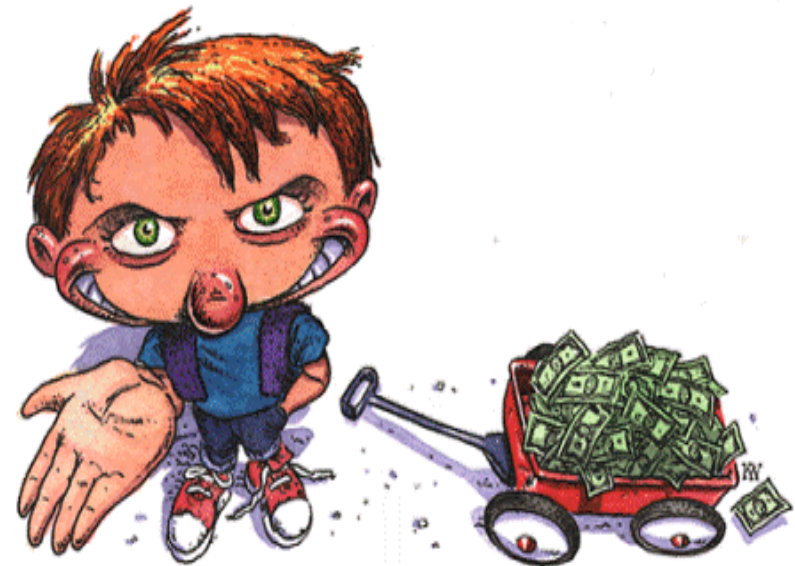
http://en.wikipedia.org/wiki/Gordon_Gekko

Greedy strategy

□ Greedy strategy

Greedy algorithms work in **stages**. In each stage, a decision is made that is good at that point, without bothering about the future. This means that some **local best** is chosen.

It assumes that a **local good** selection **makes** for a **global optimal solution**.



Greedy strategy

□ Example coin change problem

Given coins of denomination 1, 5, 10, 50\$. Find a way to change 78\$ with the fewest number of coins.

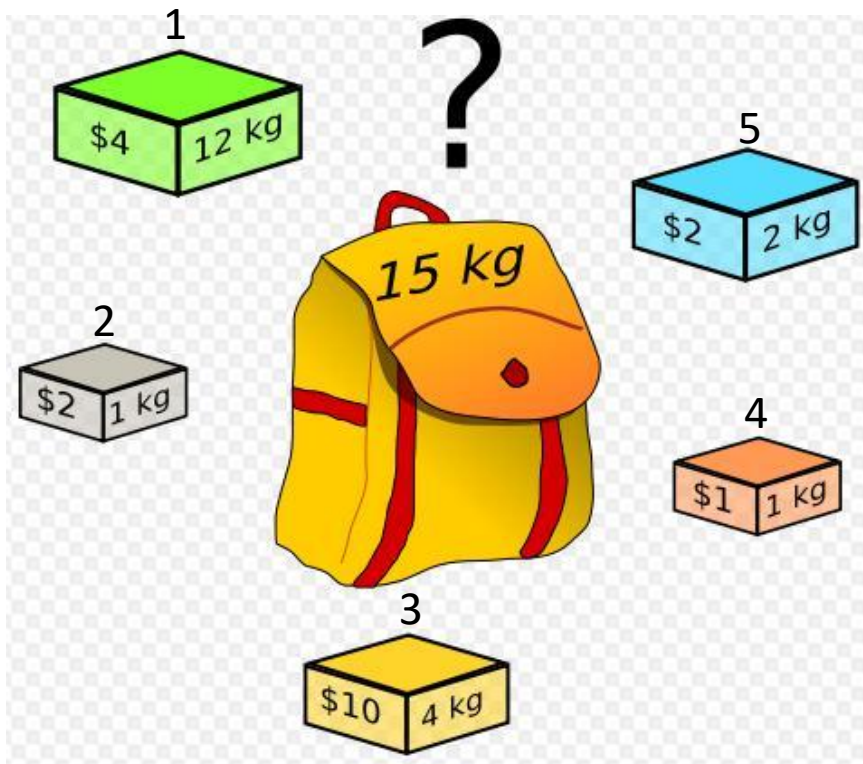
■ Using greedy strategy:

- Sort the denomination coins in decreasing order of value.
- Take the largest coin which can fit into remaining amount.

$$\begin{aligned}\Rightarrow 78 &= 1 \times 50 + 2 \times 10 + 1 \times 5 + 3 \times 1 \\ &= 7 \text{ (coins)}\end{aligned}$$

Greedy strategy

□ Example continuous knapsack problem



■ Using greedy strategy:

- Sort the items in decreasing order of unit value (value/weight).

3; 2; 5; 4; 1

- Take the valuest item which can fit into remaining bag.

$$\Rightarrow 15\text{kg} = 3 \times 4\text{kg} + 3 \times 1\text{kg}$$

3 item 3rd, 3 item 2nd

Total value: 36\$

Greedy strategy

□ Is greedy strategy always give the optimal solution?

- The answer: “No!”

- Example:

Change coin problem: denomination 10, 6, 1\$. Amount 55\$

Greedy: $55 = 5 \times 10 + 5 \times 1 \Rightarrow 10$ (coins)

Not greedy: $55 = 4 \times 10 + 2 \times 6 + 3 \times 1 \Rightarrow 9$ (coins)

$55 = 3 \times 10 + 4 \times 6 + 1 \times 1 \Rightarrow 8$ (coins)

Elements of Greedy Algorithms

- ❑ Two properties bring the effectiveness of Greedy strategy:
 - **Greedy choice property** (sự lựa chọn tham): The globally optimal solution can be obtained by making a locally optimal solution.
 - **Optimal substructure** (cấu trúc con tối ưu): An optimal solution to the problem contains optimal solutions of the subproblems.
 - ❑ To reach that effectiveness, the choice at each step must:
 - **Feasible** (khả thi): it has to satisfy the problem's constraints.
 - **Local optimal** (tối ưu cục bộ): it has to be the best local choice among all feasible choices available on that step.
 - **Irrevocable** (không thể thay đổi): once made, it cannot be changed on subsequent steps of the algorithm.
-

Greedy Template and Diagram

□ Template:

- Input: A – problem domain – Tập các đối tượng của bài toán
- Output: S - solution.

$$S = \{x_1, x_2, \dots, x_n\}; x_i \in A \text{ such } f(S) \text{ is optimal}$$

■ Functions:

- Select the best element at each stage: $x = \text{BestSelect}(A)$
 - Check if that element can be in the solution: $\text{Acceptable}(S, x)$.
 - Integrate the element to the solution: $\text{Integrate}(S, x)$.
 - Check if the solution is fully build: $\text{Is_solution}(S)$.
-

Greedy Template and Diagram

□ Diagram:

```
Greedy (A)  $\equiv$  //Optimal solution by greedy on domain A
    S =  $\emptyset$  ;
    while (A  $\neq \emptyset$  and !Is_solution(S))
        x = BestSelect(A) ;
        A = A \ {x} ;
        if (Acceptable(S, x))
            Integrate(S, x) ;
    endwhile ;
    return S ;
End.
```

Examples

-
- ✓ Change coins
 - ✓ Activity Selection Problem
 - ✓ MST with Prim, Kruskal
-

Change coins

□ Template

Input: $A = \{a_1, a_2, \dots, a_n\}$; T // a_i denomination, T amount money

Output: $S = \{k_1, k_2, \dots, k_n\}$ | // k_i number of coins a_i

□ Diagram

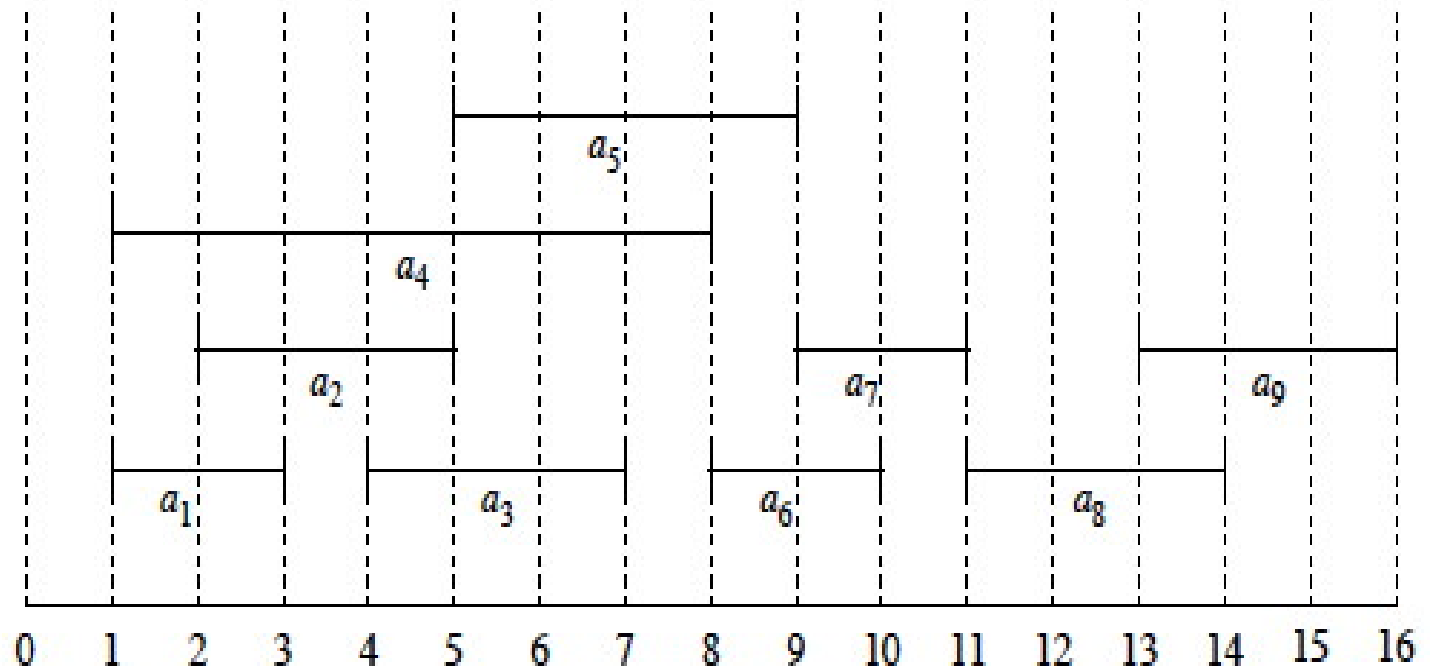
```
ChangeMoney (A, T)  $\equiv$ 
    for (i=1..n)  $k_i=0$  ;
    i=1;
    while (i<=n and T>0)
         $k_i = T/a_i$ ;  $T -= k_i * a_i$ ;
        i += 1;
    endwhile;
    if (T>0) return -1; //No solution
    else return  $\{k_1, k_2, \dots, k_n\}$ 
End.
```

Activity Selection Problem

- **Activity-selection problem:** n activities using a single resource (machine, room, etc...). Activity i take place in time from s_i (start) to f_i (finish) ($s_i < f_i$) and has fully access to the resource in the scheduled time. Find longest series of compatible activity without any overlaps?

- **Illustration:**

a_i	s_i	f_i
a_1	1	3
a_2	2	5
a_3	4	7
a_4	1	8
a_5	5	9
a_6	8	10
a_7	9	11
a_8	11	14
a_9	13	16



Activity Selection Problem

□ Problem specification:

Input: $A = \{a_i = (s_i, f_i); s_i < f_i; i=1 \dots n\}$

Output: $S = \{a_1'; a_2'; \dots a_m' \mid a_i' \in A; f_i' \leq s_{i+1}'; m - \text{max}\}$

□ Applying greedy strategy to the optimal problem

- At each step, if choose the activity finishes earliest, will have chance to choose most compatible activities follow.

□ Template

- Arranging the activities in increasing order of end time ($f_i < f_{i+1}, i=1 \dots n-1$)
- At step i ,
 - The scheduled activities $a_1'; a_2'; \dots a_{i-1}'$
 - Schedule a_i' by choosing $a_i = (s_i, f_i) \in A_i = A \setminus \{a_1'; a_2'; \dots a_{i-1}'\}$, such $s_i \geq f_{i-1}'$ and f_i is the smallest in A_i
- Start with $a_1' = a_1$

Activity Selection Problem

□ Diagram:

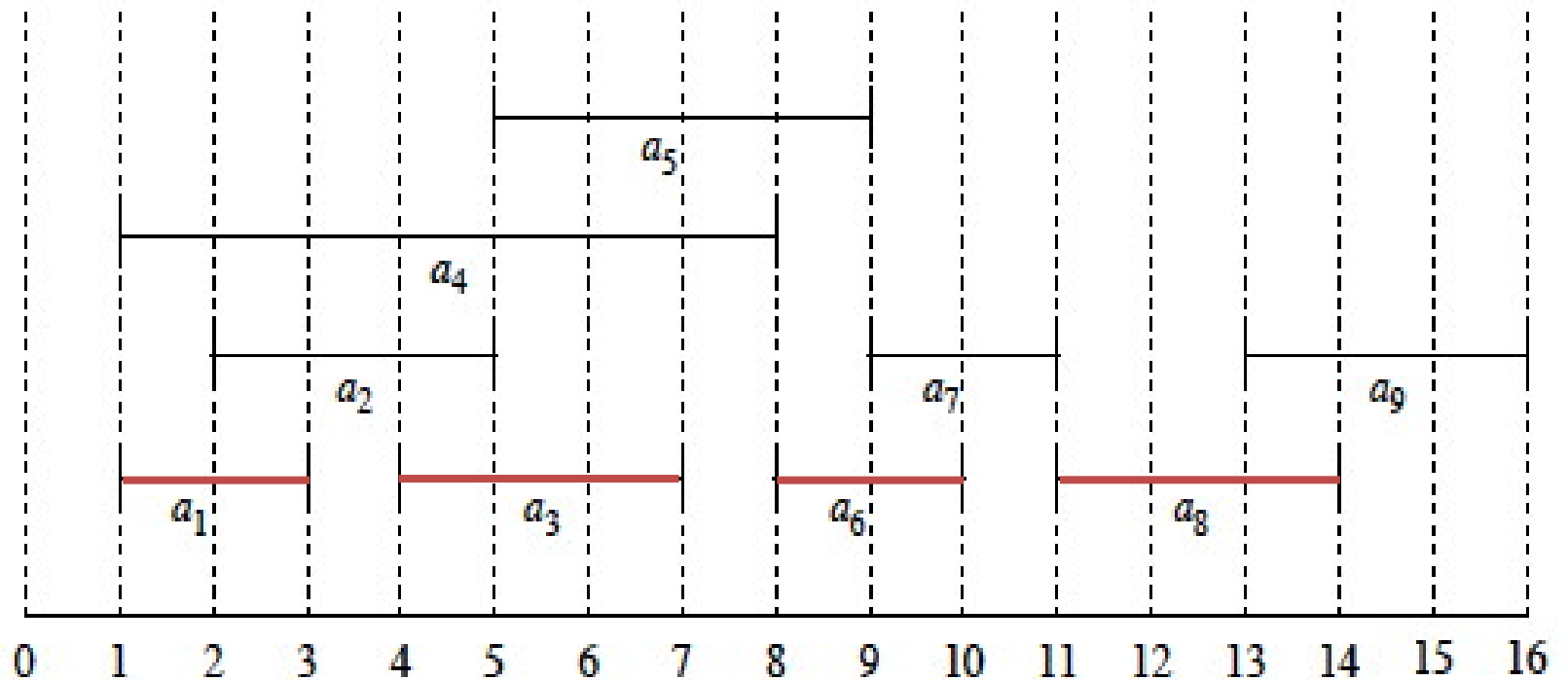
```
Activity_Selector(A) ≡  
    S={a1}  
    j=1;  
    for (i=2..n)  
        if (si≥fj)  
            S = S U {ai};    // U : concatenate to selected S  
            j = i;  
        endif;  
    endfor;  
    return S;  
End.
```

Where: $A=\{a_1, a_2, \dots, a_n\}$ are sorted in increasing order of end time ($f_i < f_{i+1}$)

Activity Selection Problem

□ Illustration:

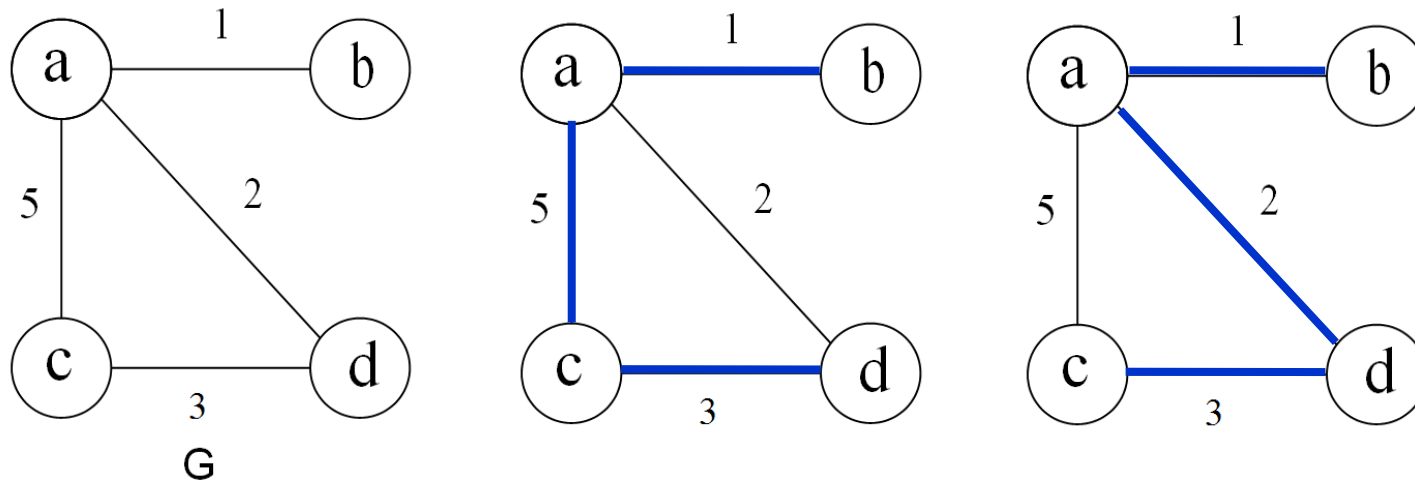
a_i	s_i	f_i
a_1	1	3
a_2	2	5
a_3	4	7
a_4	1	8
a_5	5	9
a_6	8	10
a_7	9	11
a_8	11	14
a_9	13	16



Minimum Spanning Tree Problem

□ Minimum Spanning Tree (MST) – Cây bao trùm tối thiểu

- A **spanning tree** of an undirected connected graph G is a connected acyclic subgraph (i.e., a tree) that contains all the vertices of G .
- A **minimum spanning tree** of an weighted undirected connected graph G is a spanning tree of the smallest weight.



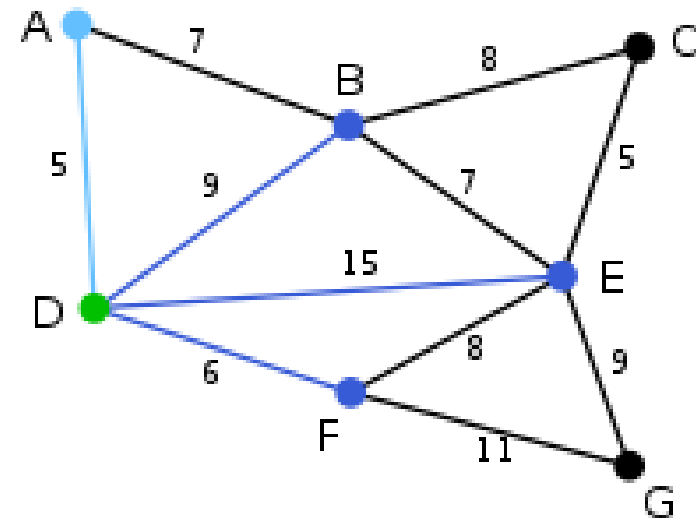
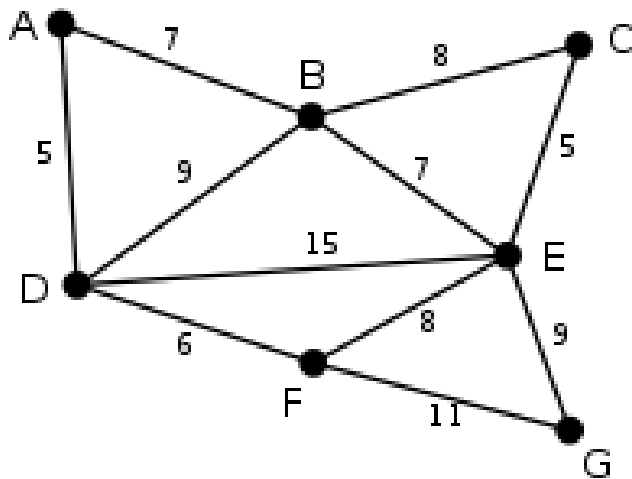
Minimum Spanning Tree Problem

□ Prim's Algorithm – Construct the MST of graph G

- Start with a subtree consists of a single vertex T_0 .
- Expand the tree by adding a vertex to the exist subtree. The expanded trees are $T_1, T_2, \dots, T_{n-1}$.
- Greedy strategy for expanding the tree at step i :
 - Simply attaching to T_{i-1} the **nearest vertex** not in that tree to get T_i .
 - The nearest vertex to T_{i-1} : a vertex not in the tree and connect to a vertex in T_{i-1} by an edge of the smallest weight.
- The algorithm stops after all the graph's vertices have been included in the tree being constructed. It is MST of G .

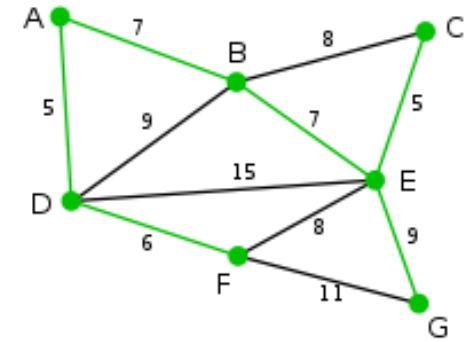
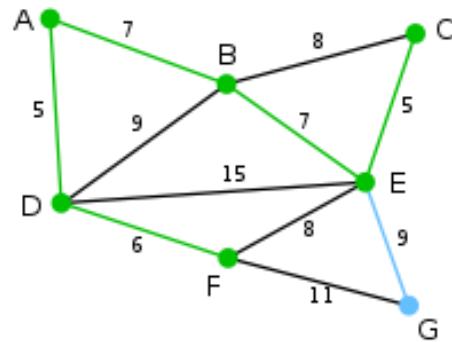
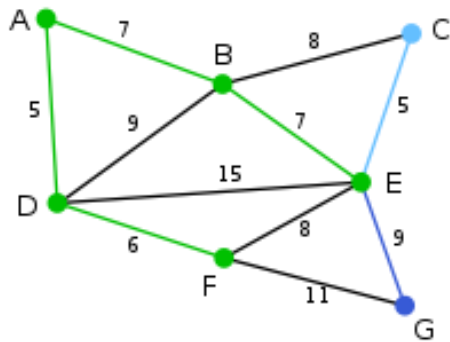
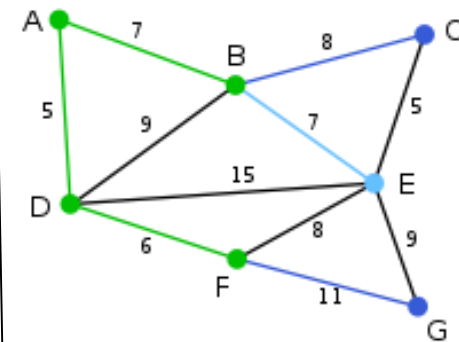
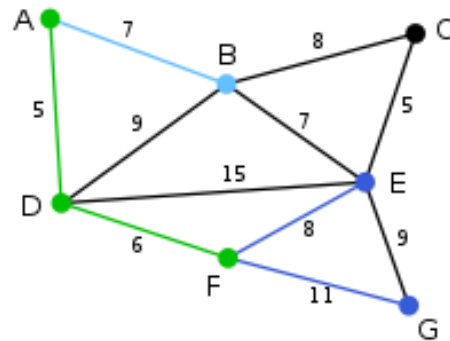
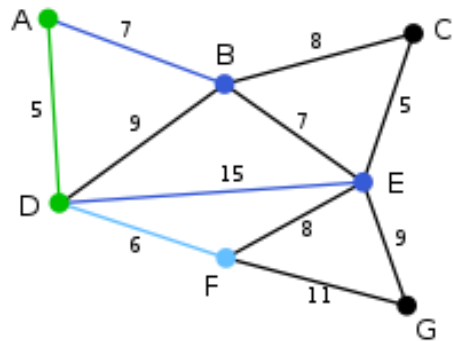
Minimum Spanning Tree Problem

□ Prim's Algorithm illustration:



Minimum Spanning Tree Problem

□ Prim's Algorithm illustration:



Minimum Spanning Tree Problem

□ Prim's algorithm diagram:

Prim(G) $\equiv$

// Input: $G = (V, E)$

// Output: E_T , set of edge of MST of G

$V_T = \{v_0\}$

$E_T = \emptyset$

for $i=1 \dots |V| - 1$

 find the minimum-weight edge $e^*=(v^*, u^*)$ among (v, u) such
 $v \in V_T$ and $u \in V - V_T$;

$V_T = V_T \cup \{u^*\}$

$E_T = E_T \cup \{e^*\}$

endfor;

return E_T ;

End.

Minimum Spanning Tree Problem

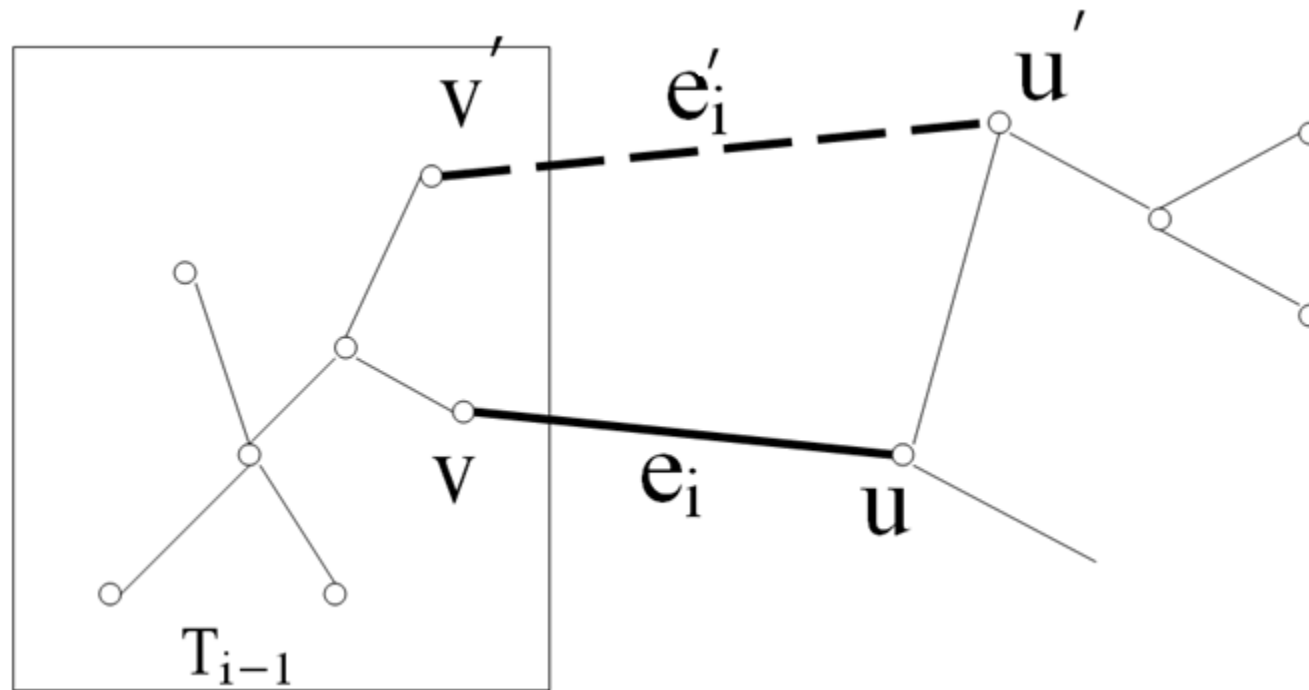
□ Prim's algorithm correctness:

Sử dụng phương pháp quy nạp và phản chứng

- Chứng minh bằng quy nạp: mỗi cây con T_i , $i = 0, \dots, n-1$, sinh ra bằng thuật toán Prim là một phần của cây bao trùm tối thiểu. Giả sử T_{i-1} là một phần của cây bao trùm tối thiểu thì T_i (sinh ra bằng cách thêm vào cạnh gần nhất) cũng là một phần của cây bao trùm tối thiểu.
- Phản chứng: giả sử cây bao trùm tối thiểu của G không chứa T_i , t.l có cạnh $e_i = (v, u)$ là cạnh có trọng số nhỏ nhất nối một đỉnh thuộc T_{i-1} với 1 đỉnh không thuộc T_{i-1} , và e_i không thuộc cây tối thiểu T (theo Prim).
- Nếu thêm e_i vào T sẽ tạo ra một chu trình, chu trình sẽ chứa cạnh $e'_i = (v', u')$ nối một đỉnh $v' \in T_{i-1}$ với 1 đỉnh $u' \notin T_{i-1}$. Nếu xóa đi cạnh (v', u') của chu trình này sẽ nhận được một cây bao trùm khác chứa e_i có trọng số nhỏ hơn T . Như vậy trái với giả thiết cây bao trùm tối thiểu không chứa $T_i \Rightarrow$ đpcm

Minimum Spanning Tree Problem

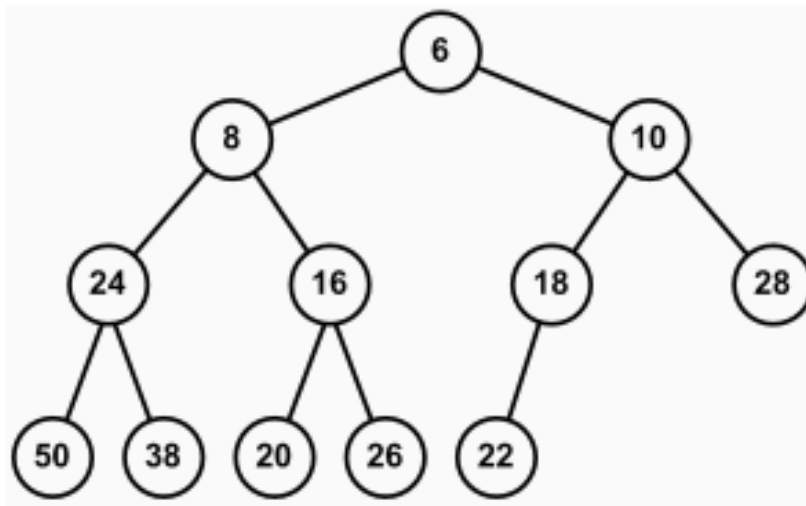
□ Prim's algorithm correctness:



Minimum Spanning Tree Problem

□ Prim's time complexity:

- Xác định đỉnh “gần nhất”:
 - Sử dụng ma trận kề biểu diễn đồ thị: Duyệt đỉnh với độ phức tạp $O(V^2)$
 - Sử dụng cây nhị phân đống nhỏ nhất (min heap) và danh sách kề biểu diễn đồ thị: $O(E \log V)$



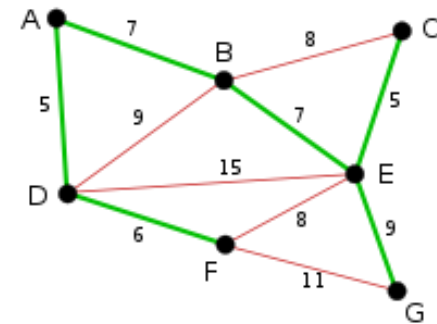
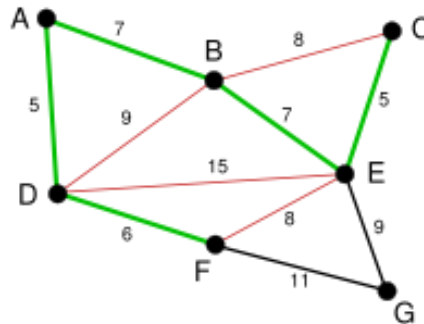
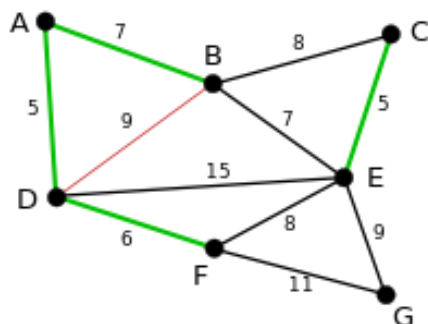
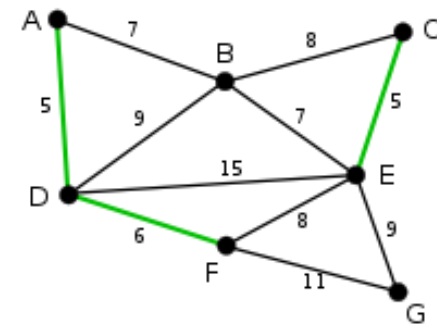
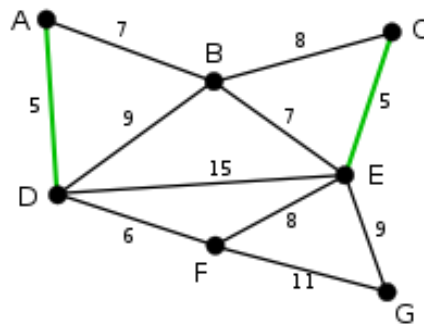
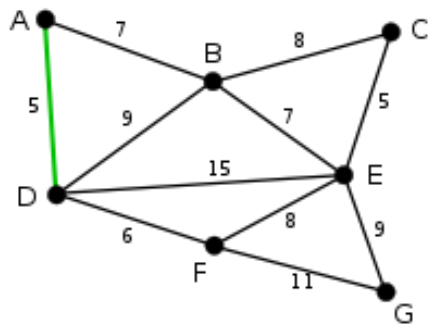
Minimum Spanning Tree Problem

□ **Kruskal's Algorithm** – Construct the MST of graph G

- Các cạnh được sắp xếp theo thứ tự tăng dần của trọng số.
 - Bắt đầu bằng một rừng (forest) rỗng.
 - Xây dựng MST theo các bước, mỗi bước thêm một cạnh
 - Trong quá trình dựng MST luôn có một “rừng”: các cây không liên thông.
 - Thêm vào cạnh có trọng số nhỏ nhất trong các cạnh chưa thêm vào cây và không tạo thành chu trình.
 - Như vậy tại mỗi bước một cạnh có thể:
 - Mở rộng một cây đã có
 - Nối hai cây thành một cây mới
 - Tạo cây mới
 - Thuật toán dừng lại khi tất cả các đỉnh đã được thêm vào.
-

Minimum Spanning Tree Problem

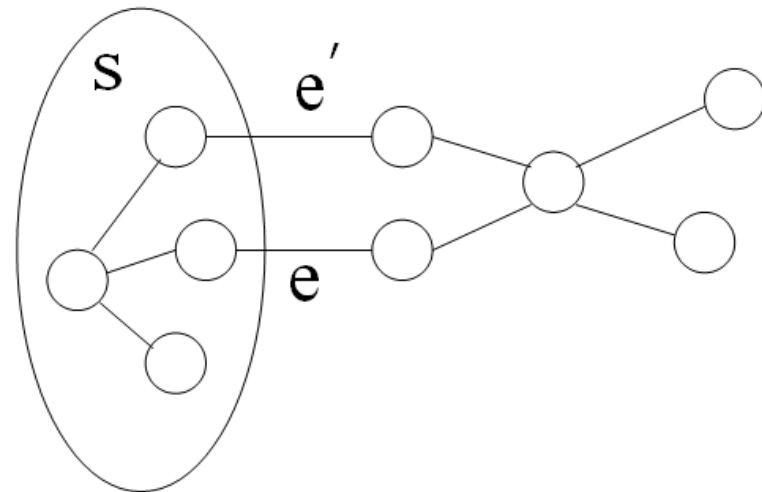
Thuật toán Kruskal



Minimum Spanning Tree Problem

❑ Thuật toán Kruskal

- Algorithm correctness:
 - The output is a spanning tree
 - The spanning tree is minimal

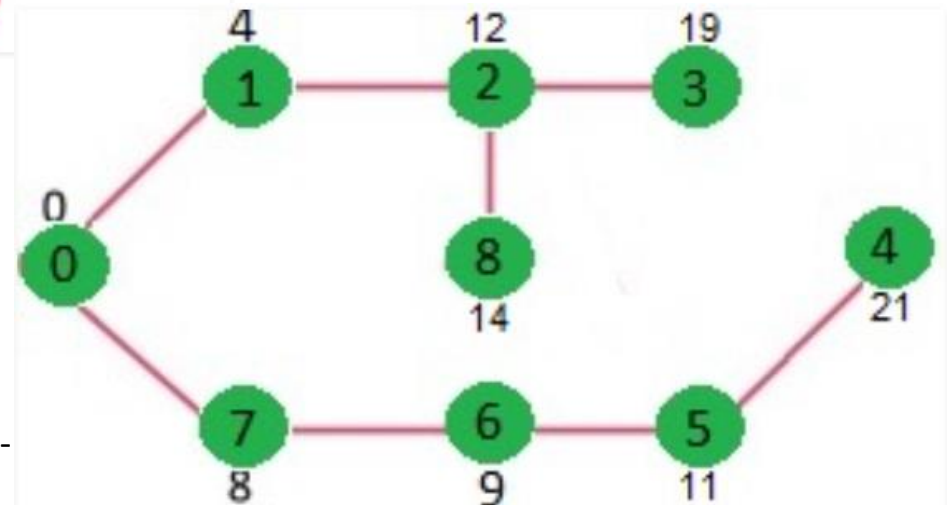
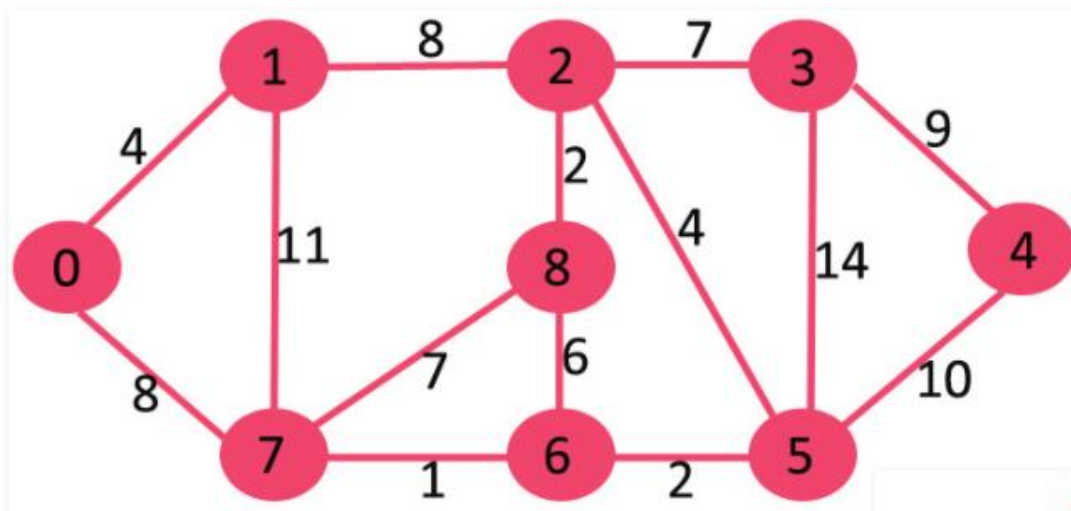


- Độ phức tạp thuật toán
 - $O(E \log E)$

Minimum Spanning Tree applications

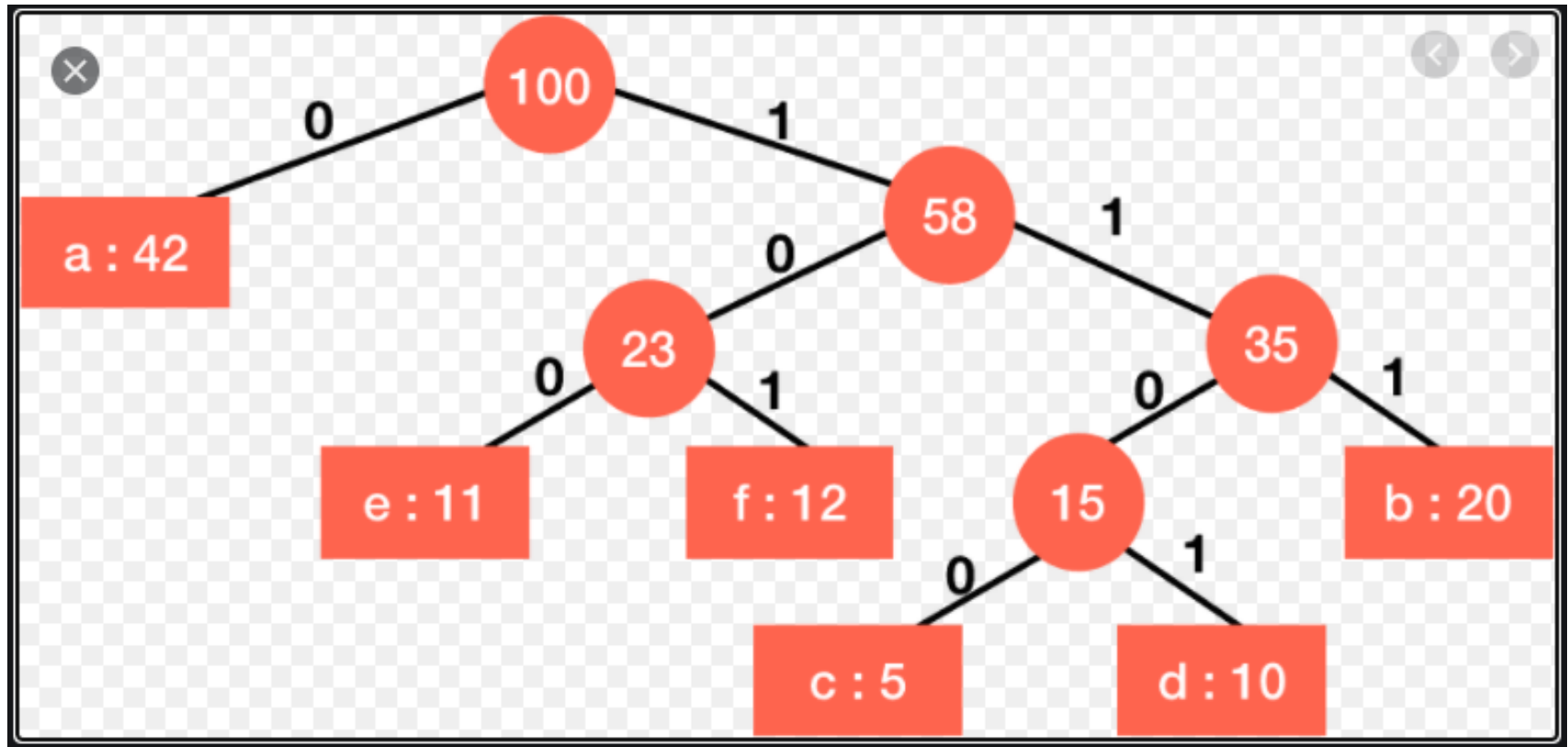
- ❑ Building a connected network
 - Computer networks, telecommunications networks
 - Transportation networks
 - Water supply networks
 - Electrical grids
 - ❑ Clustering
 - To cluster a bunch of points into k clusters
 - ❑ Traveling salesman problem
-

Dijkstra's Algorithm



<https://www.geeksforgeeks.org/dijkstras-shortest-path-algorithm-greedy-algo-7/>

Huffman code and tree



<https://www.codesdope.com/course/algorithms-huffman-codes/>

Exercises

- ❑ Anany's book chapter 9
- ❑ Other examples or applications of greedy method
- ❑ More detail in [Hw6_Greedy.doc](#)